



CS401 MPP

W3D1 Overview

Interfaces in Java 8

Overview of Today

- Java 8 Interfaces
 - Review of Enums
 - The `.forEach()` method
 - The Diamond Problem
- Overriding Methods in the Object Class
 - Overriding `.equals()`
 - Overriding `.hashCode()`

Wholeness Statement

- Java supports inheritance between classes in support of the OO concepts of inherited types and polymorphism
- Interfaces support encapsulation, play a role similar to abstract classes, and provide a safe alternative to multiple inheritance
- Likewise, relationships of any kind that are grounded on the deeper values at the source of the individuals involved result in fuller creativity of expression with fewer mistakes



CS401 MPP

Interfaces in Java 8

Interfaces in Java 8: Introduction

Java 8 Interfaces

- Before Java 8, none of the methods in an interface had a method body; all were unimplemented
- In Java 8, two kinds of implemented methods are allowed:
 - default methods
 - static methods
- Both can be added to legacy interfaces without breaking code

Default Methods

- A default method is a fully implemented method within an interface, whose declaration begins with the keyword `default`

```
public interface NameAddress {  
    String getFirstName();  
    String getLastName();  
    Address getAddress();  
  
    /** Provides a default for full name */  
    default String getFullName() {  
        return getFirstName() + " " + getLastName();  
    }  
  
    /** Provides a template for formatting output */  
    default String formattedOutput() {  
        return getFullName() + "\n" + getAddress();  
    }  
}
```

See [lesson7.lecture.defaultmethods](#)

Static Methods

- A static method in an interface is a fully implemented static method having the same characteristics as any static method in a class

```
public interface ShoppingCart {  
    String getName();  
    List<Item> getItems();  
    LocalDate getDateOfPurchase();  
  
    default double getDiscount() {  
        return 0.0;  
    }  
    static double computeBaseCost(List<Item> items) {  
        double accum = 0.0;  
        for(Item item : items) {  
            accum += (item.getPrice() + item.getTax());  
        }  
        return accum;  
    }  
    default double getTotalCost() {  
        return (1 - getDiscount()) * computeBaseCost(getItems());  
    }  
}
```

See lesson7.lecture.interfacestatic

New Programming Style

- Default Methods in an interface eliminate the need to create special classes that represent a default implementation
 - Examples from pre-Java 8 of default implementations of interfaces:
 - **WindowListener** / **WindowAdapter** (in the AWT)
 - **List** / **AbstractList**
 - Now, in developing new code, it is possible in many cases to place these default implementations in the interface directly

New Programming Style

- Static Methods in an interface eliminate the need to create special utility classes that naturally belong with the interface.
 - Examples from pre-Java 8 of how interfaces sometimes have companion utility classes (consisting of static methods):
 - **Collection / Collections**
 - **Path / Paths**
 - For new code, it is now possible to place this static companion code directly in the interface.

Solution to Evolving API

- When you need to add new methods to an existing interface, provide them with default implementations using the new Java 8 default feature.
 - Then legacy code will not be required to implement the new methods, so existing code will not be broken
 - new functionality will be available for new client projects.

Exercise 7.1

- Explore the package `exercise7_1` in the InClass Exercises project.
 - You will see a class `MyStringList` along with an interface `StringList` that it implements.
 - `StringList` contains several common list operations:
 - `String[] strArray(); int size(); void setSize(int s);`
 - `void add(String s); String get(int i);`
- Use Java 8 default methods to make implementations of `add` and `get`
 - This considerably reduces the effort to implement `StringList` in `MyStringList` since most of the implementation work has been moved into the interface

Something like this could have been done in Java's `List` interface except that the `List` interface was created long before default interface methods were introduced.



CS401 MPP

Interfaces in Java 8

Review of Enums

Review of Enums

- An enumerated type is a Java class all of whose possible instances are explicitly enumerated during initialization

```
public enum Size { SMALL, MEDIUM, LARGE };  
  
//usage  
if(requestedSize == Size.LARGE) {  
    applyDiscount();  
}
```

The enum `Size` (a special kind of Java class) has been declared to have just three instances: `SMALL`, `MEDIUM`, `LARGE`

Use Enums as:

Enums were added in Java 1.5

Before that people used constants

1. Constants in an application

- Old / Weak Programming Practice:
 - Create a class (or interface) containing constants, stored as public static final values
 - Problem: No compiler control over usage of these constants when they occur as input arguments to methods (example on next slide)
- Better Approach: represent constants as instances of an enum(erated) type

2. Optimal, thread-safe implementation of the Singleton Pattern

Both code pieces from java.awt.Label (AWT is the oldest Java GUI)

Example of Constants in Java

```
package java.awt;
...
public class Label extends Component implements Accessible {
...
    /**
     * Indicates that the label should be left justified.
     */
    public static final int LEFT = 0;

    /**
     * Indicates that the label should be centered.
     */
    public static final int CENTER = 1;

    /**
     * Indicates that the label should be right justified.
     * @since JDK1.0t.
     */
    public static final int RIGHT = 2;

    public Label(String text, int alignment)
        throws HeadlessException {
        GraphicsEnvironment.checkHeadless();
        this.text = text;
        setAlignment(alignment);
    }
}
```

```
...
/**
 * Sets the alignment for this label to the specified alignment.
 * Possible values are <code>Label.LEFT</code>,
 * <code>Label.RIGHT</code>, and <code>Label.CENTER</code>.
 * @param alignment the alignment to be set.
 * @exception IllegalArgumentException if an improper value for
 *         <code>alignment</code> is given.
 * @see java.awt.Label#getAlignment
 */
public synchronized void setAlignment(int alignment) {
    switch (alignment) {
        case LEFT:
        case CENTER:
        case RIGHT:
            this.alignment = alignment;
            LabelPeer peer = (LabelPeer)this.peer;
            if (peer != null) {
                peer.setAlignment(alignment);
            }
            return;
    }
    throw new IllegalArgumentException("improper alignment: " + alignment);
}
```

Use of constants was flawed,
but the only way before enums

Problem with Constants

- No compiler control over use of constants.

```
Label label = new Label("Hello", 23);
```

Compiler is fine with this, 23 is an int...
just not one we're expecting

- The compiler sees the correct type
 - at runtime 23 will be recognized as an illegal value
- Enums are compiler checked
 - Only Objects from that enum have the correct type

Improved Label using Enum

```
package lesson7.lecture.enums.mylabel;
```

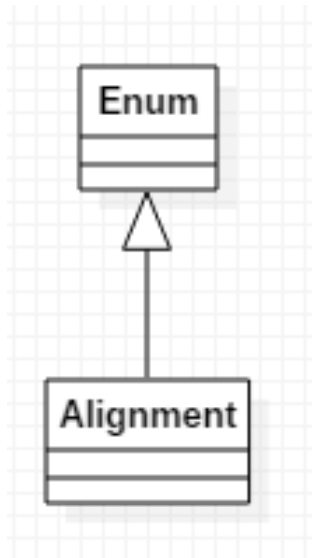
```
public enum Alignment {  
    LEFT, CENTER, RIGHT;  
}
```

See: lesson7.lecture.enums.*

```
public class Label {  
    private String text;  
    private Alignment alignment;  
  
    public Label(String text, Alignment alignment) {  
        this.text = text;  
        setAlignment(alignment);  
    }  
  
    public Alignment getAlignment() {  
        return alignment;  
    }  
  
    public synchronized void setAlignment(Alignment alignment) {  
        this.alignment = alignment;  
    }  
}
```

Enums Inherit from Enum

- In the Label example (previous slide)
 - Each of the instances of the Alignment enum has type Alignment
 - Which is a subclass of Enum
 - Alignment itself is a class
 - Alignment cannot inherit anymore
 - (multiple inheritance not allowed).



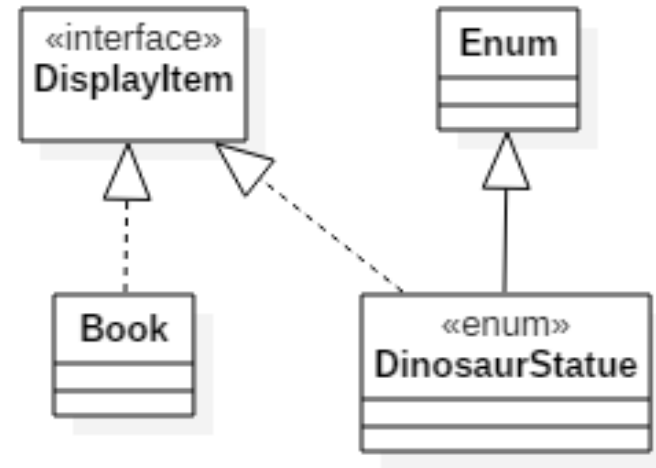
In Java 8 Enums can “Inherit”

```
public interface DisplayItem {  
    public default String displayInfo() {  
        ...  
    }  
}
```

```
public class Book implements DisplayItem {  
    ...  
}
```

```
public enum DinosaurStatue implements DisplayItem {  
    INSTANCE;  
}
```

see: [lesson7.lecture.enums3.java7](#)
and: [lesson7.lecture.enums3.java8](#)



Need a Specific Type?

- What if you have constants that must be of a specific type, like int or String (or another type)?

Don't create constants!

Create an enum that holds data!

```
public class DimConstants {  
    public static final double LENGTH = 1.0;  
    public static final double WIDTH = 2.0;  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(DimConstants.LENGTH);  
    }  
}
```

```
public enum Dim {  
    LENGTH(1.0), WIDTH(2.0);  
  
    private double val;  
  
    private Dim(double val) {  
        this.val = val;  
    }  
  
    public double val() {  
        return this.val;  
    }  
}
```

Just like a normal class,
you can have as many instance
variables as you like

```
public class Test {  
    public static void main(String[] args) {  
        System.out.println(Dim.LENGTH.val());  
    }  
}
```

Enum Singletons

- The singleton design pattern
 - Restricts instantiation of a class to a single object
 - Is easy to implement with an Enum

see: lesson7.lecture.singletons

```
public enum MySingleton {  
    INSTANCE;  
  
    public void behavior() {  
        System.out.println("hello");  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        MySingleton.INSTANCE.behavior();  
    }  
}
```

Access it like this

Enum Best Practice

- From Bloch, Effective Java (2nd edition):
 - Use enums (in place of public static final variables) whenever you need a fixed set of constants all of whose values you know at compile time

Exercise 7.2

- Replace the code below with an **enum Const**
 - Make sure it provides the same functionality
 - Be sure to update the main method to test it
 - Starter code in the **InClassExercises** project

```
package lesson7.exercise_2;

public class Constants {
    public static final String COMPANY = "Microsoft";
    public static final int SALES_TARGET = 200000000;
}
```



CS401 MPP

Interfaces in Java 8

forEach

Iterable Interface (Java 7)

- The Iterable interface is part of the Collections API
 - Implementing Iterable allows a class to be used with the “for each” construct
 - implemented by all collection classes to support iteration through a collection
 - The only method in Iterable is iterator(), which returns an Iterator
 - The Iterator Interface has two methods:
 - hasNext()
 - next()

```
System.out.println("-----using for each-----");  
for (Object object : list) {  
    System.out.println(object.toString());  
}
```

See: lesson7.lecture.iterator

.forEach()

- Java 8 adds the **default** method **forEach** to the Iterable interface
 - And thereby to every class that implements Iterable
 - forEach takes a consumer to act on each element

```
Consumer<String> consumer = new Consumer<String>() {  
    @Override  
    public void accept(String s) {  
        System.out.println(s);  
    }  
};  
System.out.println("-----using new forEach method-----");  
list.forEach(consumer);
```

See: lesson7.lecture.iterator

Implementation

- The **forEach** method applies the **accept** method of **Consumer** to each element

```
default void forEach(Consumer<? super T> action) {  
    Objects.requireNonNull(action);  
    for (T t : this) {  
        action.accept(t);  
    }  
}
```

- **Consumer** is a new interface in Java 8

```
public interface Consumer<T> {  
    void accept(T input);  
}
```

Just one abstract method
that accepts a single argument
and produces void

Exercise 7.3

- Use the new Java 8 `forEach` method on a Java list to copy all its elements into the instance of `MyStringList`
 - You'll start with a Java `ArrayList` containing multiple elements and an empty `MyStringList` (from Exercise 7.1)
 - Startup code is in the `lesson7.exercise_3` package of the `InClassExercises` project
 - Test your work by using the `main` method in the `ListInfo` class



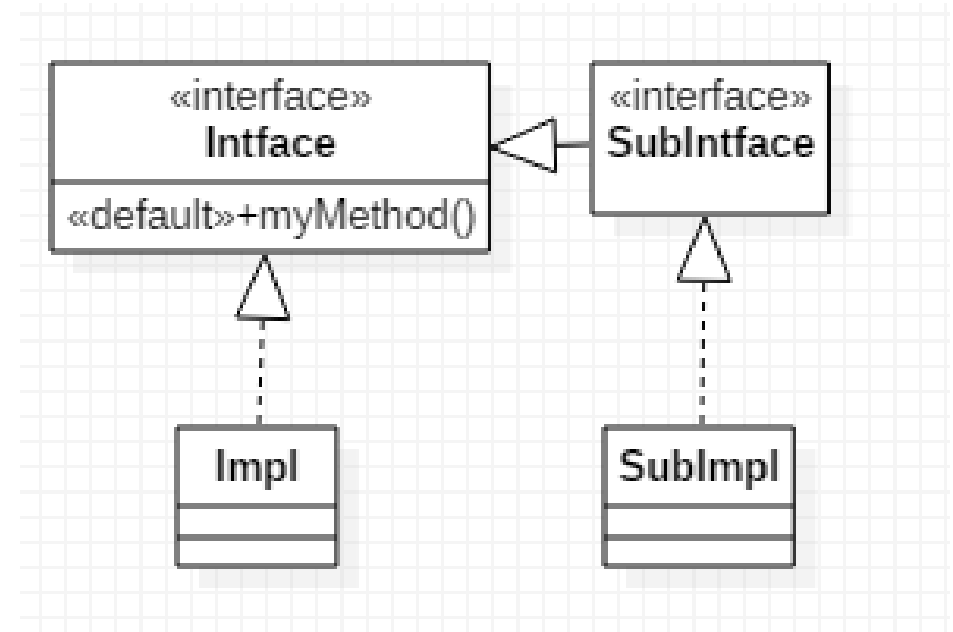
CS401 MPP

Interfaces in Java 8

The Diamond Problem

2 Types of Clash Possible

- Potential clash if:
 - two interfaces have the same method
 - one interface and a superclass have the same method
- No clash if a class implements an interface with a default method
 - That class inherits the default method
 - Or can override it



Interfaces Clash

- There are two ways that interfaces can clash
 1. A class implements 2 interfaces
 - Both have a method with the same signature
 - One of these is a default method
 2. A (sub)interface extends 2 interfaces
 - Both have a method with the same signature
 - One of these is a default method

Interface Clash Solutions

- To solve “a class Implements 2” clash:
 - Either override the method
 - Or declare it abstract
- To solve “an interface extends 2” clash:
 - Either write a new default methods in this interface
 - Or just declare the method (and it defaults to abstract)

Basically the same solution
in both situations:

Provide your own implementation,
or declare it as abstract

Clash: Superclass VS Interface

- Superclass wins!
 - When a class extends a superclass and also implements an interface with a default method
 - If there is a conflict, the superclass implementation wins
 - The class gets the method inherited from the superclass
 - The subclass / implementer is not required to do anything (like overriding or declaring abstract)

See: `lesson7.lecture.defaultmethodrules`

Static Methods Do Not Clash

- Static methods in an interface are not inherited by implementers
 - Therefore there is no clash if two interfaces implement static methods with the same signature
 - This is different from the behavior for subclasses of a class (they do inherit static methods)

See: `lesson7.lecture.interfacestatic_clash`

Exercise 7.4

- Look at the 3 code snippets on the PDF file in `lesson7.exercise_4` package of the `InClassExercises` project.
- Try to determine, without using a compiler, what happens when the code is compiled / run.

Main Point 1

- Interfaces are used to specify publicly available services in the form of method declarations
 - A class that implements such an interface must make each of the methods operational
 - Interfaces may be used polymorphically, in the same way as a superclass in an inheritance hierarchy
 - Because many interfaces can be implemented by the same class, interfaces provide a safe alternative to multiple inheritance
 - Java8 now supports static and default methods in an interface, which make interfaces even more flexible: For instance, enums can now “inherit” from other types and new public operations can be added to legacy interfaces without breaking code (example: `forEach` method in the `Iterable` interface)
- The concept of an interface is analogous to the creation itself – the creation may be viewed as an “interface” to the undifferentiated field of pure consciousness; each object and avenue of activity in the creation serves as a reminder and embodiment of the ultimate reality



CS401 MPP

Interfaces in Java 8

FPP Review

Overriding Methods in Object

- The Object class is the superclass of all Java classes
 - It contains several useful methods

Most of the time only really useful
if they are overridden

- These methods are:

`.toString()`

`.equals()`

`.hashCode()`

Overriding toString()

- The purpose of toString() is to provide a String representation of the state of an object

```
// Example from FPP
@Override
public String toString() {
    return "Account type: " + acctType +
        ", balance:" + balance;
}
```

Which can be logged or
printed to the console

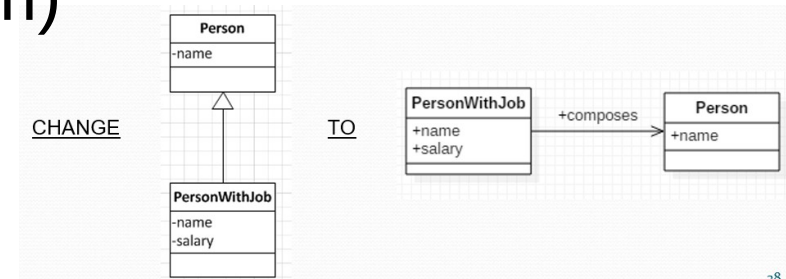
Best Practice: For every significant class
you create, Override the toString method

Overriding equals()

- Care is needed in overriding equals when using inheritance (say B extends A)
- If it is acceptable for B to use the same equals method as used in A, then the best strategy is the **instanceof** strategy and make **equals final**.

Overriding equals()

- If two different equals methods are required, two strategies are possible
 - Use the **same classes strategy**, but declare **subclass B to be final**
 - Use **composition** instead of inheritance – this will always work as long as inheritance between B and A is not needed (for polymorphism)



Overriding hashCode()

- Two general rules for creating hash codes:
 1. Equal keys must be given the same hash code
 - ✗ Otherwise same key will occupy different slots in the table
 2. Different keys should be given different hash codes
 - ✗ Otherwise keys will go to the same slot in the table

Best Practice: The hash codes should be distributed as evenly as possible
(each integer appears as a hash code as frequently as any other)

Overriding hashCode()

- Best practices:
 - Whenever equals is overridden, hashCode should also be overridden
 - The hashCode method should take into account the same fields as the equals method
 - The class on which the object is based should be immutable

To define your own hashCode method, use `Objects.hash(...)`

Example

```
public class Person {  
    private String name;  
    private int age;  
    private LocalDate hireDate;  
    @Override  
    public int hashCode() {  
        return Objects.hash(age, hireDate, name);  
    }  
    @Override  
    public boolean equals(Object obj) {  
        if (this == obj)  
            return true;  
        if (obj == null)  
            return false;  
        if (getClass() != obj.getClass())  
            return false;  
        Person other = (Person) obj;  
        return age == other.age  
            && Objects.equals(hireDate, other.hireDate)  
            && Objects.equals(name, other.name);  
    }  
}
```

Objects.hash() can take
any number of arguments

Review: Immutable

- A class is immutable if the data it stores cannot be modified once it's been initialized.
- Java's String and number classes (Integer, Double, BigInteger, etc) are immutable.
- Immutable classes provide good building blocks for creating more complex objects
- Java 8 LocalDate is also immutable

Review: Immutable

- Immutable classes tend to be smaller and focused (building blocks)
- If many instances are needed a mutable companion should also be created (like `StringBuilder`) to handle multiplicities without hindering performance

Immutability Guidelines

From Effective Java 2nd edition

- All fields should be private and final
 - This keeps internals private and prevents data changing
- Provide getters but no setters for all fields
 - No setters because there should be no change
- Make the class final
 - Another way to prevent accessing internals
- Make sure that getters don't return mutable objects



CS401 MPP

Overriding Methods in Object

Closing Remarks

Connecting the Parts of Knowledge With the Wholeness of Knowledge

1. Inheritance in Java makes it possible for a subclass to enjoy (and re-use) the features of a superclass.
 2. All classes in Java – even user defined classes – automatically inherit from the class Object
-
3. Transcendental Consciousness is the field of pure awareness, beyond the active thinking level, that is the birthright and essential nature of everyone. Everyone “inherits” from pure consciousness
 4. Wholeness moving within itself: In Unity Consciousness, there is an even deeper realization: The only data and behavior that exist in the universe is that which is “inherited from” pure consciousness – everything in that state is seen as the play of one’s own consciousness.





CS401 MPP

FPP Review

Inherit from Object

The Object Class

- Singly-rooted inheritance tree: every Java class extends Object
 - Directly or indirectly
- No explicit mention of extends is needed
 - Your class extends Object if you don't extend anything
- Every class has the following methods (and others not cover here):
 - `public String toString()`
 - `public boolean equals(Object o)`
 - `public int hashCode()`
 - `protected Object clone() throws CloneNotSupportedException`

toString()

- If a class does not override the default implementation given by the Object class

```
public class Person {  
    private String name;  
  
    public Person(String n) {  
        name = n;  
    }  
  
    public static void main(String[] args) {  
        Person p = new Person("Joe");  
        System.out.println(p);  
    }  
}
```

Prints: tmp.Person@6d06d69c

This is a concatenation of the fully qualified class name
With the hex version of the hash code / memory address

Overriding toString()

- Allows you to capture the state of the current object and send it to a log file or to the console
 - Log files can help solve a problem after release
 - Printing to console helps debug during development

```
public class Person {  
    private String name;  
    public Person(String n) { name = n; }  
  
    @Override  
    public String toString() {  
        return "Person [name=" + name + " ]";  
    }  
  
    public static void main(String[] args) {  
        System.out.println(new Person("Joe"));  
    }  
}
```

Note the @Override

Prints: Person [name=Joe]

Automatic calls to toString()

- `toString()` is automatically called when you pass an object to `System.out.println` or include it in the formation of a `String`

```
Account acct = new Account("Savings", 500.00);  
String output = "The account " + acct;  
System.out.println(acct);
```

toString for Arrays

- Suppose we have the array:

```
String[] people = {"Bob", "Harry", "Sally"};
```

- Wrong way to make a String:

```
people.toString();
```

- Right way to make a String:

```
Arrays.toString(people);
```

Overriding `.equals()`

- Default implementation is same as for `==`
 - `obj1.equals(obj2)` if and only if `obj1 == obj2`
 - Same if and only if references point to same object
- What if we have a Person class?

```
public class Person {  
    protected String name;  
    public Person(String n) { name = n; }  
  
    public static void main(String[] args) {  
        Person p1 = new Person("Joe");  
        Person p2 = new Person("Joe");  
        System.out.println(p1.equals(p2));  
    }  
}
```

Two person objects should equal
each other if they have the same data
But they don't!

Correct way

```
@Override
public boolean equals(Object aPerson) {
    if (aPerson == null) return false;
    if (!(aPerson instanceof Person)) return false;
    Person p = (Person) aPerson;
    boolean isEqual = this.name.equals(p.name);
    return isEqual;
}
```

- The argument must be of type object (else compiler error)
- If aPerson is null there is no way we're equal (we're not null)
- If aPerson is not a Person (or subclass), no way we're equal
- After these special cases, we check that the data (here just name) is the same and declare sameness based on that

InstanceOf / Same Class

- Last slide we used InstanceOf, another strategy is to check that the class is the same

```
@Override
public boolean equals(Object aPerson) {
    if (aPerson == null) return false;
    if (aPerson.getClass() != this.getClass()) return false;
    Person p = (Person) aPerson;
    boolean isEqual = this.name.equals(p.name);
    return isEqual;
}
```

The same-classes-strategy
for overriding equals

Potential Problem with Same-Classes Strategy

- If a subclass of Person is introduced
 - subclass inherits the equals method
 - but it always returns false when comparing a superclass instance with a subclass instance

```
public class PersonWithJob extends Person {  
    private double salary;  
    public PersonWithJob(String n, double s) {  
        super(n);  
        salary = s;  
    }  
}
```

See: [lesson7.lecture.overrideequals.equalclassesstrategyXX](#)

Potential Problem with Same-Classes Strategy

```
public class Person {  
    protected String name;  
    public Person(String n) {  
        name = n;  
    }  
    @Override  
    public boolean equals(Object aPerson) {  
        if(aPerson == null) return false;  
        if(aPerson.getClass() != this.getClass()) return false;  
        Person p = (Person)aPerson;  
        boolean isEqual = this.name.equals(p.name);  
        return isEqual;  
    }  
    public static void main(String[] args) {  
        Person p1 = new Person("Joe");  
        Person p2 = new PersonWithJob("Joe", 20000);  
        //Prints false to console in each case  
        System.out.println("p1.equals(p2)? " + p1.equals(p2));  
        System.out.println("p2.equals(p1)? " + p2.equals(p1));  
    }  
}
```

For this reason, whenever same-classes strategy is used, you should either:

1. declare the superclass final (to prevent subclassing) OR
2. override equals separately in the subclass (if desired)

Potential Problem With InstanceOf Strategy

- If a subclass of **Person** is introduced
 - The subclass inherits the **equals** method
 - If subclass overrides **equals**, then an *asymmetric equals* is created

```
public class PersonWithJob extends Person {  
    private double salary;  
    public PersonWithJob(String n, double s) {  
        super(n);  
        salary = s;  
    }  
}
```

See: [lesson7.lecture.overrideequals.instanceofstrategyXX](#)

Potential Problem With InstanceOf Strategy

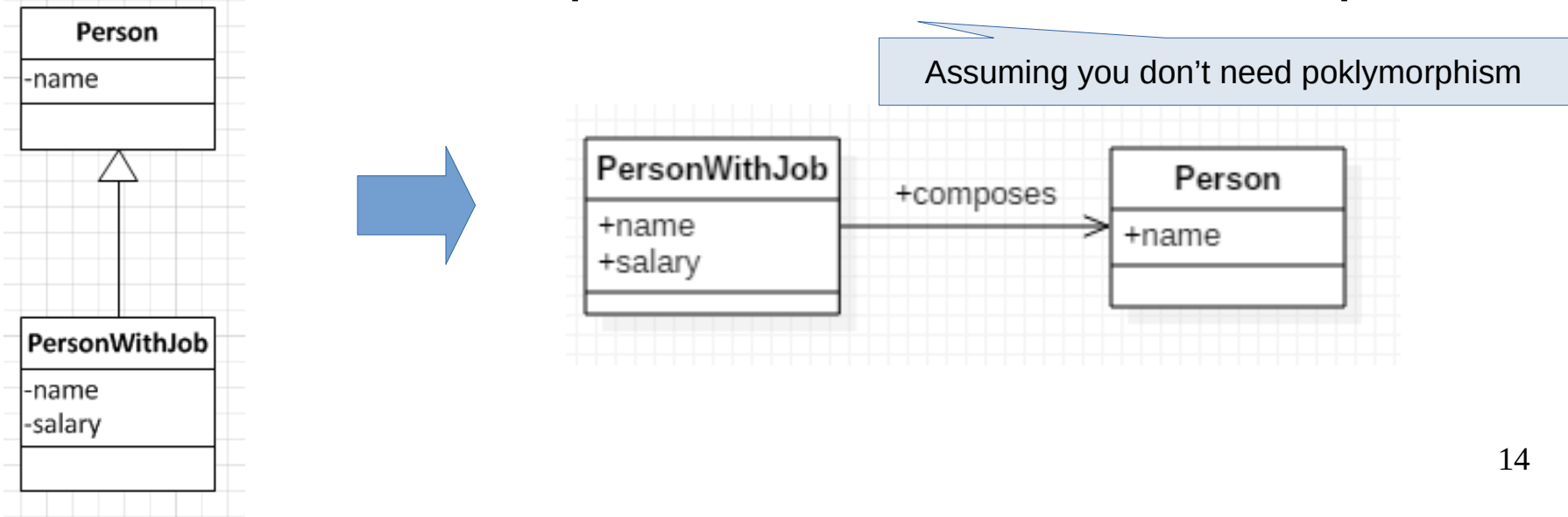
```
public class Person {  
    protected String name;  
    public Person(String n) {  
        name = n;  
    }  
  
    public boolean equals(Object aPerson) {  
        if(aPerson == null) return false;  
        if(!(aPerson instanceof Person)) return false;  
        Person p = (Person)aPerson;  
        boolean isEqual = this.name.equals(p.name);  
        return isEqual;  
    }  
    public static void main(String[] args) {  
        Person p1 = new Person("Joe");  
        Person p2 = new PersonWithJob("Joe", 20000);  
        //First output is true, second output is false  
        System.out.println("p1.equals(p2)? " + p1.equals(p2));  
        System.out.println("p2.equals(p1)? " + p2.equals(p1));  
    }  
}
```

For this reason, whenever instance-of strategy is used, you should either:

1. declare the superclass final (to prevent subclassing) OR
2. require that every subclass relies on the superclass version of equals() (and does not override equals() separately)

Composition instead of Inheritance

- Either approach (equal-classes or instanceof strategy) is error-prone
 - Safe alternative: replace inheritance with composition



Best Practices for equals()

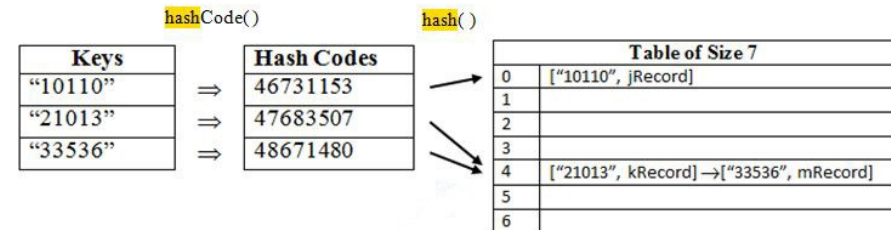
- Suppose B extends A
 - If it's acceptable for B to use the same equals method as used in A: use **instanceof strategy**
 - Otherwise:
 - Use same classes strategy and make subclass B final
 - Use composition instead of inheritance

Assuming you don't need polymorphism

hashCode()

- Java uses the **hashCode** method available on a class to transform a key (of that class) into a small integer
 - The integer serves as an index in an underlying array
 - Used when objects of any kind (Integers, Strings, chars, or any others) are used as keys in a hashtable / hashmap

User's View of Hashtable	
Key (= Employee ID)	Value (= Record)
"10110"	jRecord
"21013"	kRecord
"33536"	mRecord



Default hashCode ()

- Default implementation of hashCode() (provided by the Object class) is generally not useful
 - Just based on the memory address of the object

```
public class Main {  
    public static void main(String[] args) {  
        Person p1 = new Person("Joe", "Smith", 100000, new GregorianCalendar(1988, 5, 5));  
        Person p2 = new Person("Anne", "Jones", 80000, new GregorianCalendar(1986, 3, 9));  
  
        HashMap h = new HashMap();  
        Pair key1 = new Pair(p1.getFirstName(), p1.getLastName());  
        Pair key2 = new Pair(p2.getFirstName(), p2.getLastName());  
  
        h.put(key1, p1);  
        h.put(key2, p2);  
  
        Pair lookup = new Pair("Joe", "Smith");  
        System.out.println(lookup.equals(key1));  
        System.out.println(h.containsKey(lookup));  
        System.out.println(h.get(lookup));  
    }  
}
```

Prints:
false
false
null

Overriding hashCode ()

- The default **hashCode()** method fails to take into account the instance variables used by the **equals ()** method to determine that two Person objects are equal
- Whenever **equals ()** is overridden:
 - **hashCode ()** should also be overridden!
 - The **hashCode ()** method should take the same fields into account as the equals method

Overriding hashCode()

- Two general rules for creating hash codes:
 1. Equal keys must be given the same hash code
 - ✗ Otherwise same key will occupy different slots in the table
 2. Different keys should be given different hash codes
 - ✗ Otherwise keys will go to the same slot in the table

Best Practice: The hash codes should be distributed as evenly as possible
(each integer appears as a hash code as frequently as any other)

Legacy Approach to hashCode()

- Build the hash code from the hash codes of each of the instance variables in the class
 - If `f` is boolean do `(f ? 1 : 0)`
 - If `f` is a byte, char, short, or int do `(int) f`
 - If `f` is a long do: `(int) (f ^ (f >>> 32))`
 - If `f` is a float do: `Float.floatToIntBits(f)`
 - If `f` is a double do:
`long f1 = Double.doubleToLongBits(f);`
`(int) (f1 ^ (f1 >>> 32))`
 - If `f` is an object use `f.hashCode()`

Best if `f` has own `.hashCode()` method

Legacy Approach

- Then combine the hashcodes to produce a final hash code.
- Say you have instance variables u, v, w
 - Obtain the hashes for them as previously described
 - Then put them together

```
@Override
public int hashCode() {
    // create hash_u, hash_v, hash_w

    int result = 17;
    result += 31 * result + hash_u;
    result += 31 * result + hash_v;
    result += 31 * result + hash_w;
    return result;
}
```

Modern VS Legacy

- When implementing hashCode() for the Pair class

```
@Override
public int hashCode() {
    return Objects.hash(first, second);
}
```

Modern way is to just pass all your
Instance variables to Objects.hash()

```
@Override
public int hashCode() {
    int result = 17; // seed
    int hashFirst = first.hashCode();
    int hashSecond = second.hashCode();
    result += 31 * result + hashFirst;
    result += 31 * result + hashSecond;
    return result;
}
```



CS401 MPP

Fpp Review

Nested Classes

Member Inner Classes

```
public class Member {  
    private String name = "Joe";  
    private Pair p = new Pair();  
    {  
        p.first = 4;  
        p.second = 5;  
        System.out.println(p);  
    }  
    private void printHello() {  
        System.out.println("Hello " + name);  
    }  
    class Pair {  
        int first;  
        int second;  
        Pair() {  
            printHello();  
        }  
        public String toString() {  
            return "(" + first + ", " + second + ")";  
        }  
    }  
    public static void main(String[] args) {  
        new Member();  
    }  
}
```

- Accessible only when an instance of enclosing class exists, and when inner class has been instantiated
- Have access to private members of enclosing class
- May be declared private, public, etc
- Cannot contain static variables or methods
- Enclosing class can access private members of inner class, relative to an existing reference
- Best Practice: Should be accessed only by the enclosing class

Static Nested Classes

```
public class Static {  
    private String name = "Joe";  
    private Pair p = new Pair();  
    {  
        p.first = 4;  
        p.second = 5;  
        System.out.println(p);  
    }  
    private void printHello() {  
        System.out.println("Hello" + name);  
    }  
    static class Pair {  
        int first;  
        int second;  
        Pair() {  
            //no access  
            //printHello();  
        }  
        public String toString() {  
            return "(" + first + ", " + second + ")";  
        }  
    }  
    public static void main(String[] args) {  
        (new Static()).printHello();  
    }  
}
```

- Considered a "top-level class", packaged inside another class
- May be instantiated even if enclosing class has not been instantiated
- No access to non-static members of enclosing class without a reference (with a reference, can access private members)
- Direct access to static members
- May be declared private, public, etc
- May contain static and non-static members
- Enclosing class can access private members of nested class, relative to an existing reference

Local Inner Classes

```
public class Local {  
    private String name = "Joe";  
    public void printPair(int x, int y) {  
        class Pair {  
            int first;  
            int second;  
            Pair() {  
                printHello(name);  
            }  
            public String toString() {  
                return "(" + first + ", "  
                    + second + ")";  
            }  
        }  
        Pair p = new Pair();  
        p.first = x;  
        p.second = y;  
        System.out.println(p);  
    }  
    private void printHello(String n) {  
        System.out.println("Hello " + n);  
    }  
    public static void main(String[] args) {  
        (new Local()).printPair(11, 3);  
    }  
}
```

- Always defined within a method body – never accessible from outside the method in which it is defined
- Has access to all members of enclosing class
- Has access to local variables, but may not modify them (they are "effectively final")
- Access specifiers (public, private...) may not be used in definition of a local inner class
- Local inner classes provide strong encapsulation – no other method in the enclosing class (or anywhere else) can access it.

Anonymous Inner Classes

```
public class Anonymous {  
    interface IPair {  
        public void printHello();  
    };  
  
    private String name = "Joe";  
    public void printPair(int x, int y) {  
        (new IPair() {  
            int first = x;  
            int second = y;  
  
            public String toString() {  
                return "(" + first + ", " +  
                    second + ")";  
            }  
            public void printHello() {  
                name = "Tom";  
                System.out.println("Hello " +  
                    name + "\n" + this);  
            }  
        }).printHello();  
    }  
    public static void main(String[] args) {  
        (new Anonymous()).printPair(11, 3);  
    }  
}
```

- Defines and instantiates, at the same time, a class, without giving it a name
- The syntax can be used to define a subclass of a given class or an implementation of a given interface
- Main usage: when class definition involves few lines of code and the class needs to be defined only once (example: attaching a handler to a button)